PS3 Predictive Analytics

Barni (bfb.eco@cbs.dk)

```r
library(fpp3)
```

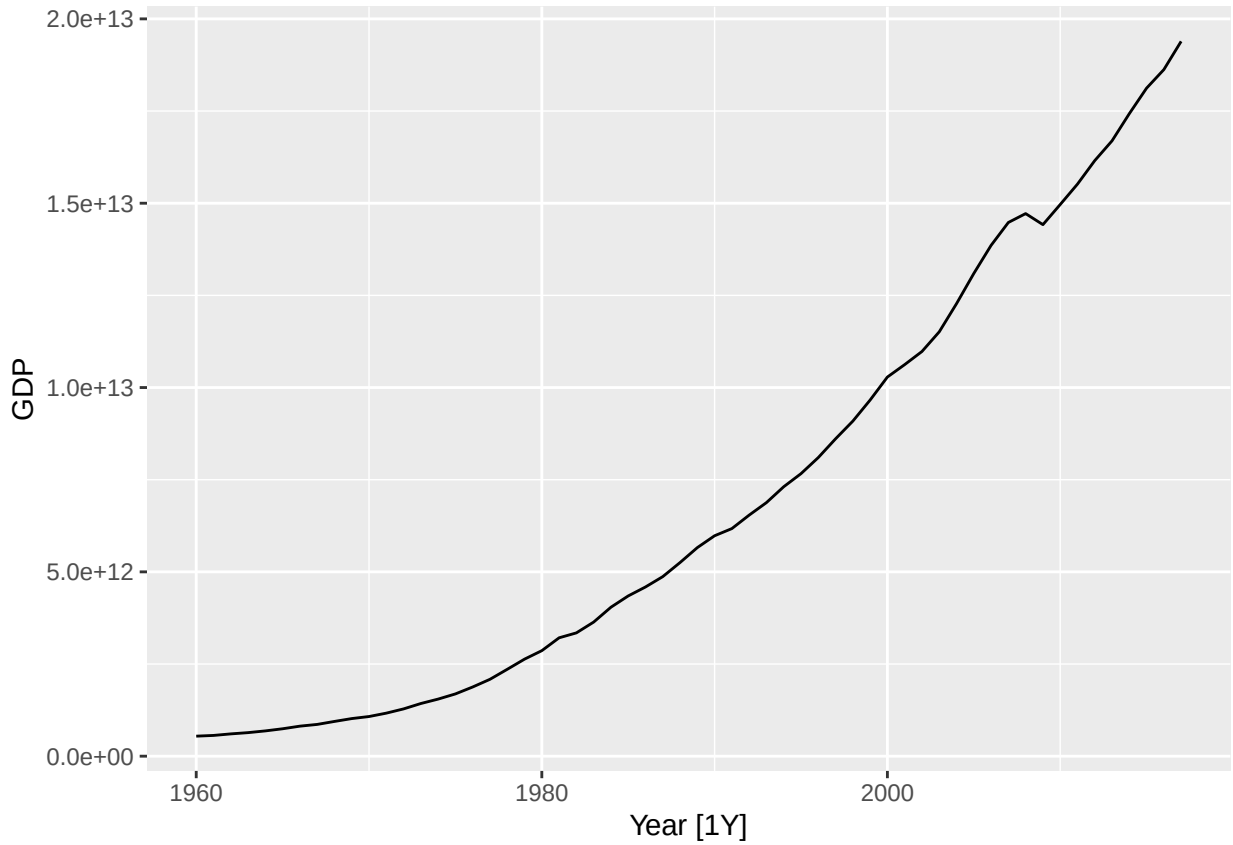
```
## -- Attaching packages ---------------------------------------------- fpp3 0.5 --

## v tibble      3.2.1     v tsibble     1.1.4
## v dplyr       1.1.4     v tsibbledata 0.4.1
## v tidyr       1.3.0     v feasts      0.3.1
## v lubridate   1.9.3     v fable       0.3.3
## v ggplot2     3.4.4     v fabletools  0.3.4

## -- Conflicts ------------------------------------------------- fpp3_conflicts --
## x lubridate::date()    masks base::date()
## x dplyr::filter()      masks stats::filter()
## x tsibble::intersect() masks base::intersect()
## x tsibble::interval()  masks lubridate::interval()
## x dplyr::lag()         masks stats::lag()
## x tsibble::setdiff()   masks base::setdiff()
## x tsibble::union()     masks base::union()
```

3.2

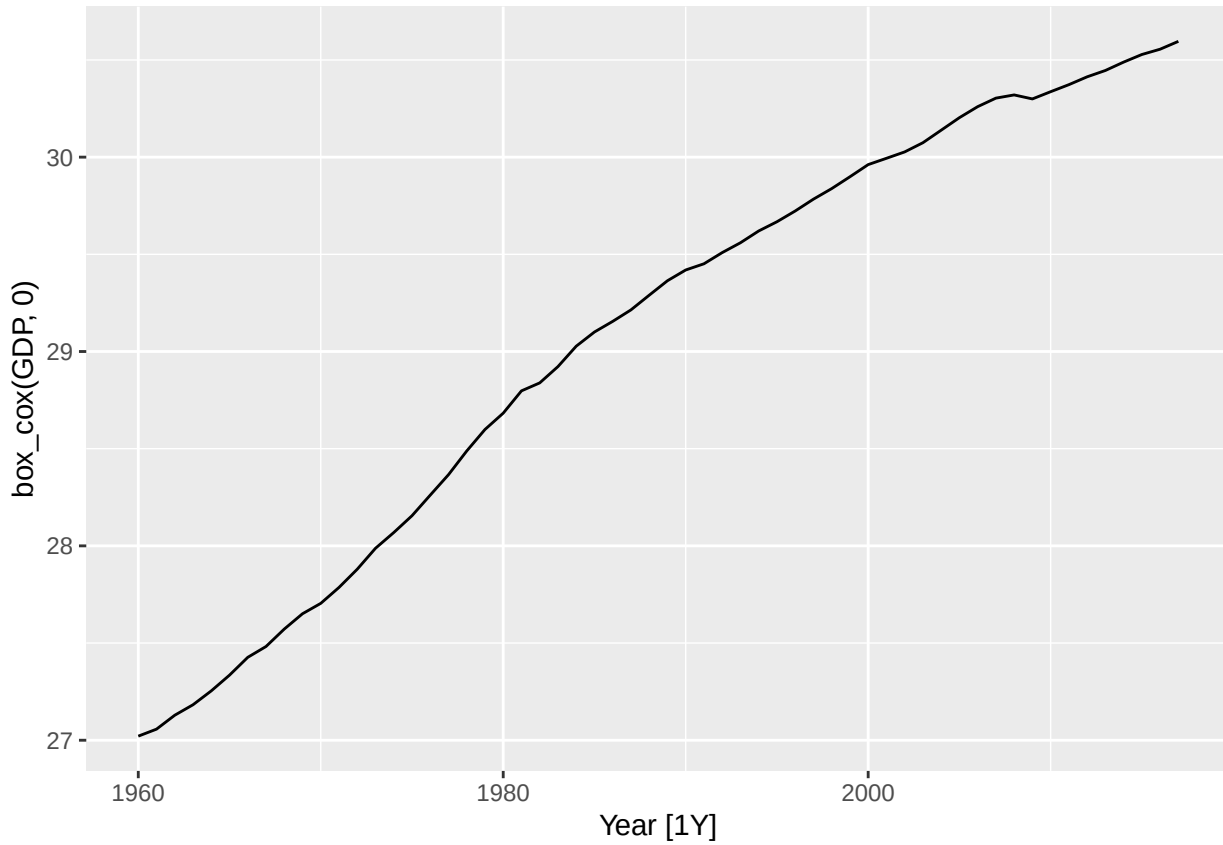
For each of the following series, make a graph of the data. If transforming seems appropriate, do so and describe the effect.

- United States GDP from *global_economy*
- Slaughter of Victorian "Bulls, bullocks and steers" in *aus_livestock*
- Victorian Electricity Demand from *vic_elec*.
- Gas production from *aus_production*

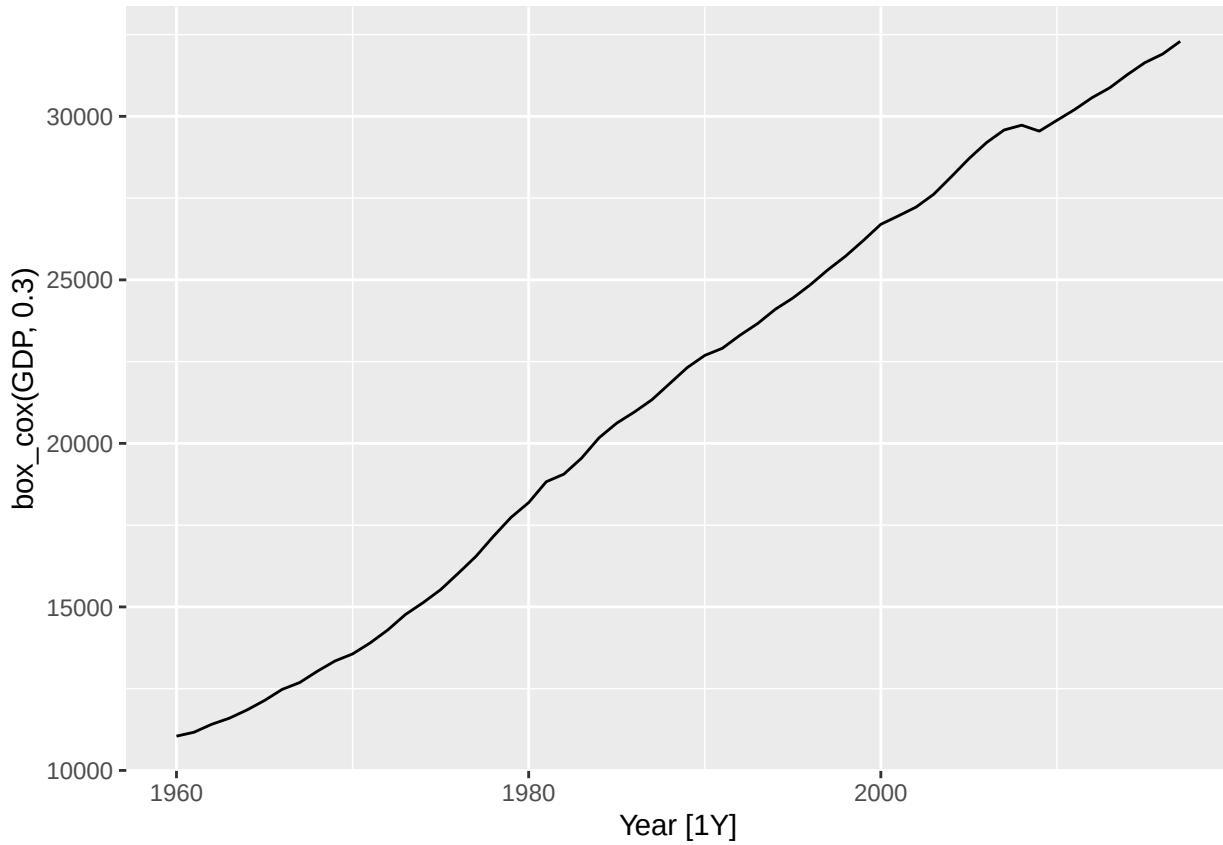
```r
# United States GDP

us_economy <- global_economy %>%
  filter(Country == "United States")
us_economy %>%
  autoplot(GDP)
```




```r
# Trend appears exponential, a transformation would be useful.

us_economy %>%
  autoplot(box_cox(GDP, 0))
```



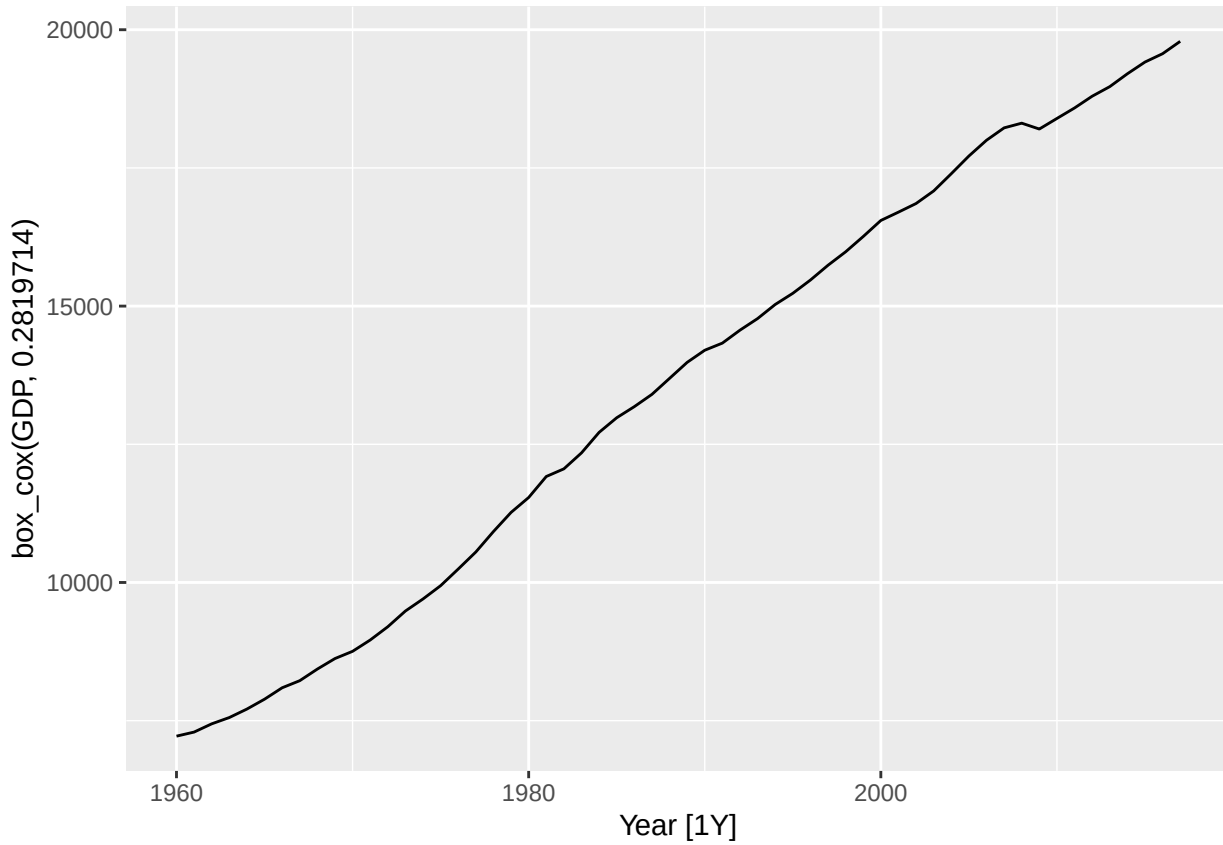

```
# A log transformation (Box-Cox with lambda=0) appears slightly too strong.

us_economy %>%
  autoplot(box_cox(GDP, 0.3))
```




```
#Using lambda=0.3 looks pretty good, the trend is now almost linear.

# Let's see what guerrero's method suggests.

us_economy %>%
  features(GDP, features = guerrero)
```

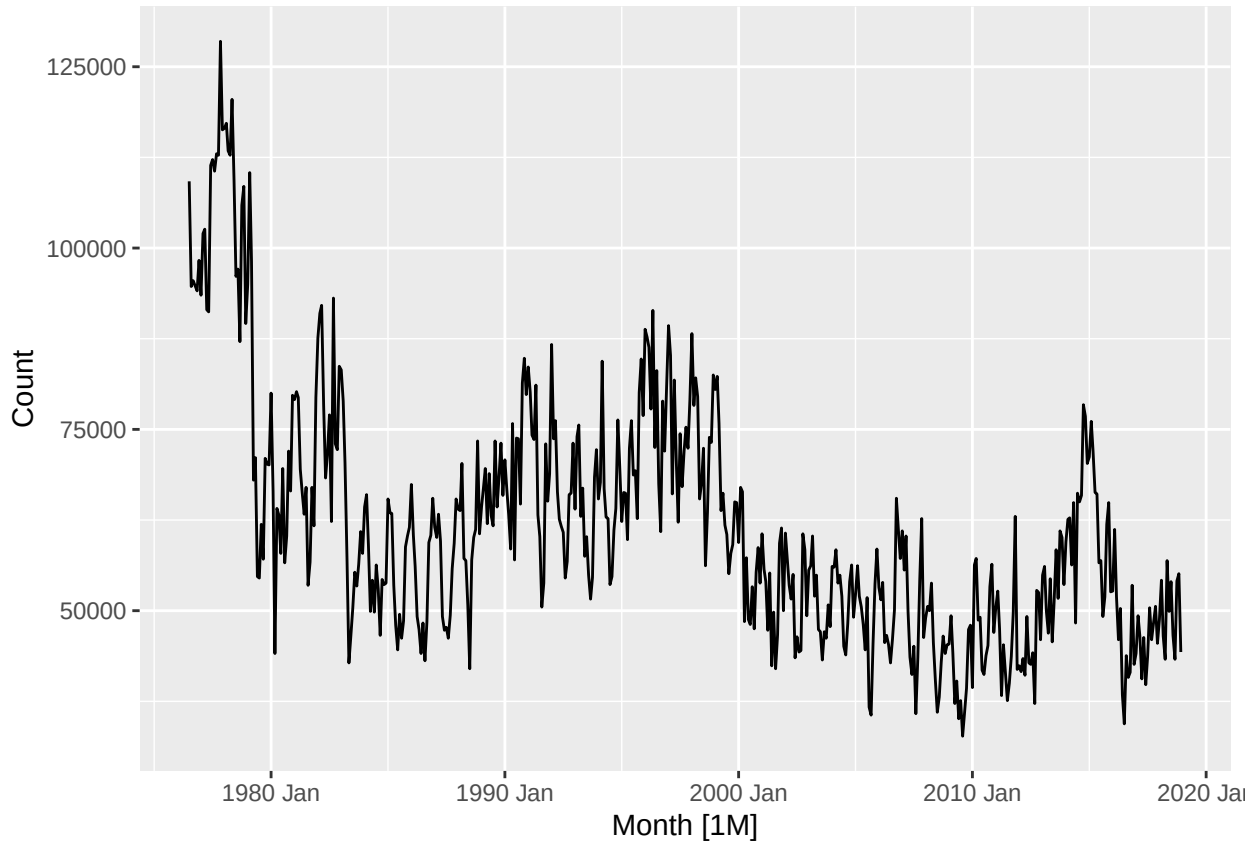
```
## # A tibble: 1 x 2
##   Country       lambda_guerrero
##   <fct>                   <dbl>
## 1 United States           0.282
```

```
#Pretty close to lambda=0.3, let's see how it looks:

us_economy %>%
  autoplot(box_cox(GDP, 0.2819714))
```

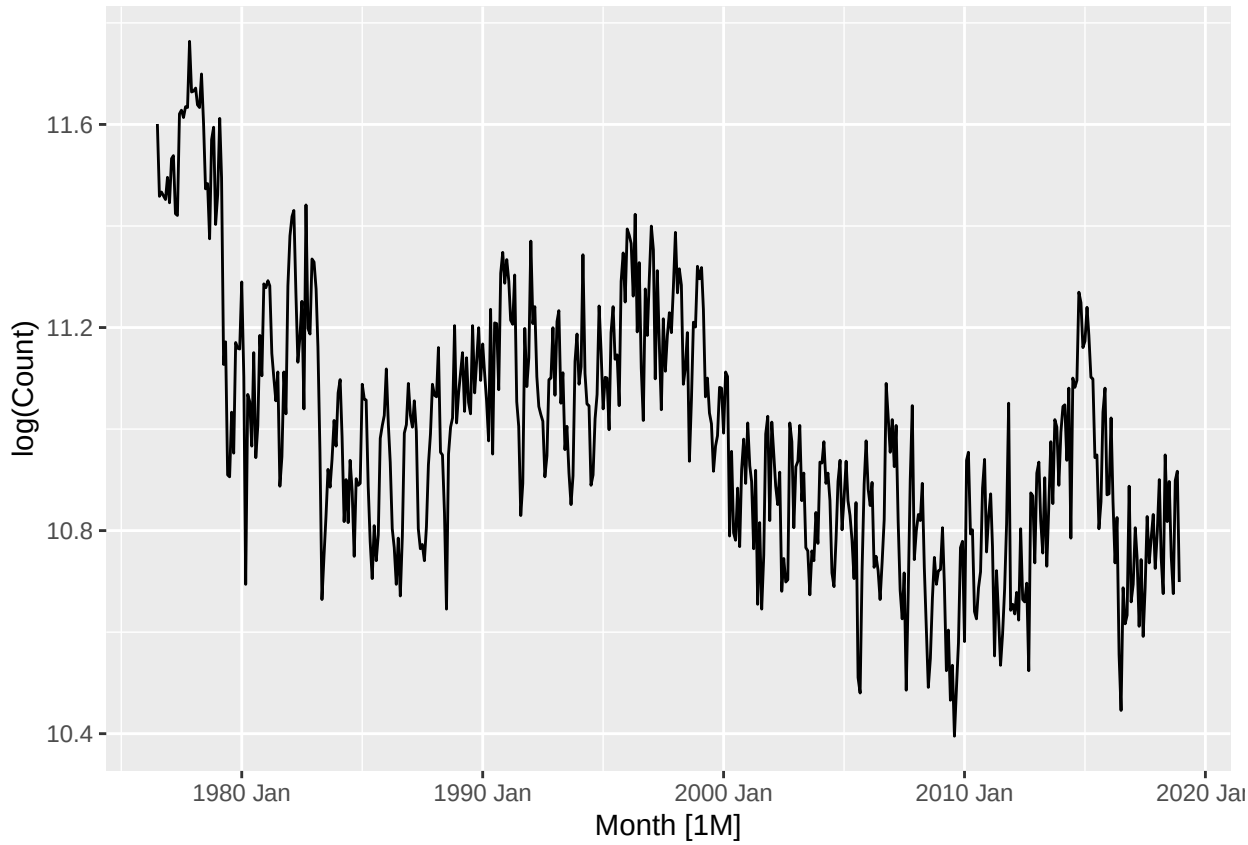



```
# More or less the same. Box-Cox transformations are usually insensitive to
# the choice of lambda
```

```
# Slaughter of Victorian "Bulls, bullocks and steers"

vic_bulls <- aus_livestock %>%
  filter(State == "Victoria", Animal == "Bulls, bullocks and steers")
vic_bulls %>%
  autoplot(Count)
```




```
# Variation in the series appears to vary slightly with the number of bulls
# slaughtered in Victoria. A transformation may be useful.

vic_bulls %>%
  autoplot(log(Count))
```

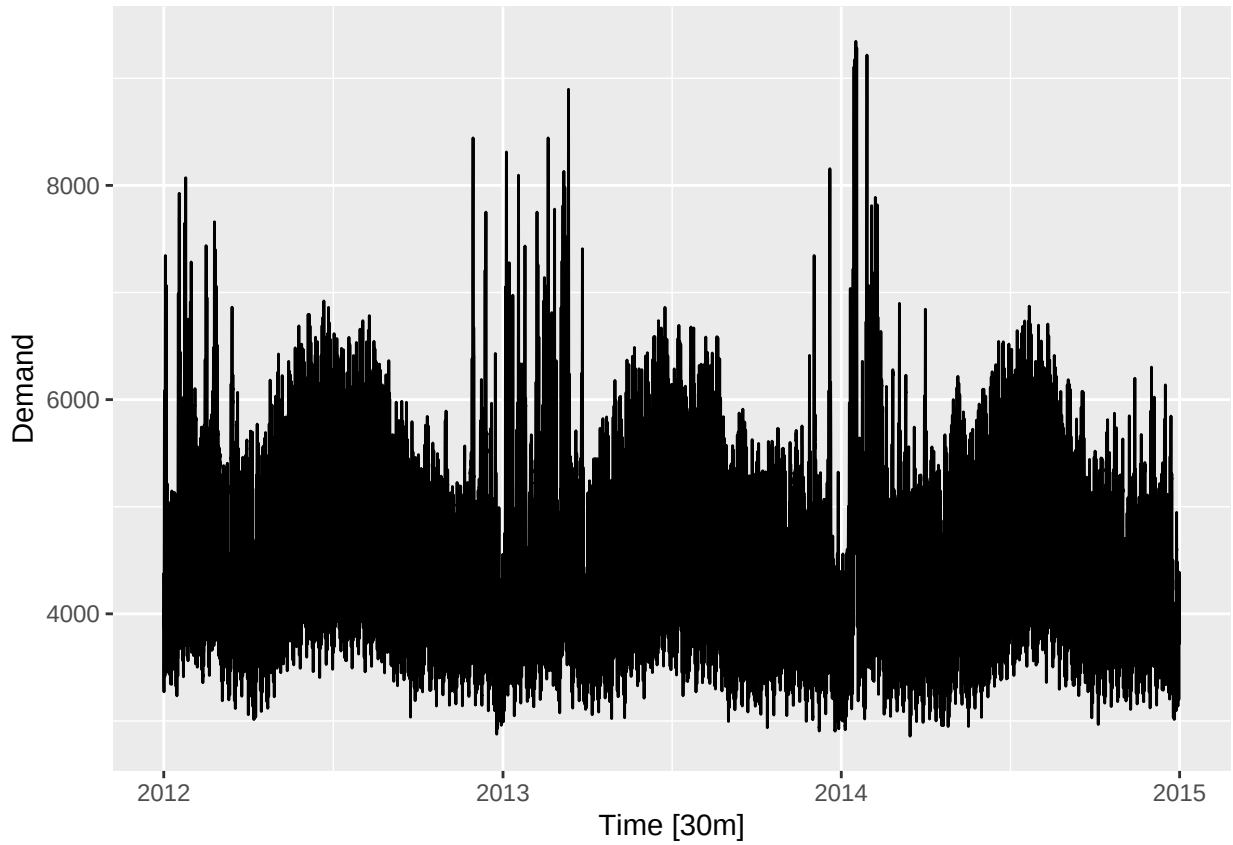



```
# A log transformation (Box-Cox lambda=0) appears to normalise most of the
# variation. Let's check with guerrero's method.

vic_bulls %>%
  features(Count, features = guerrero)
```

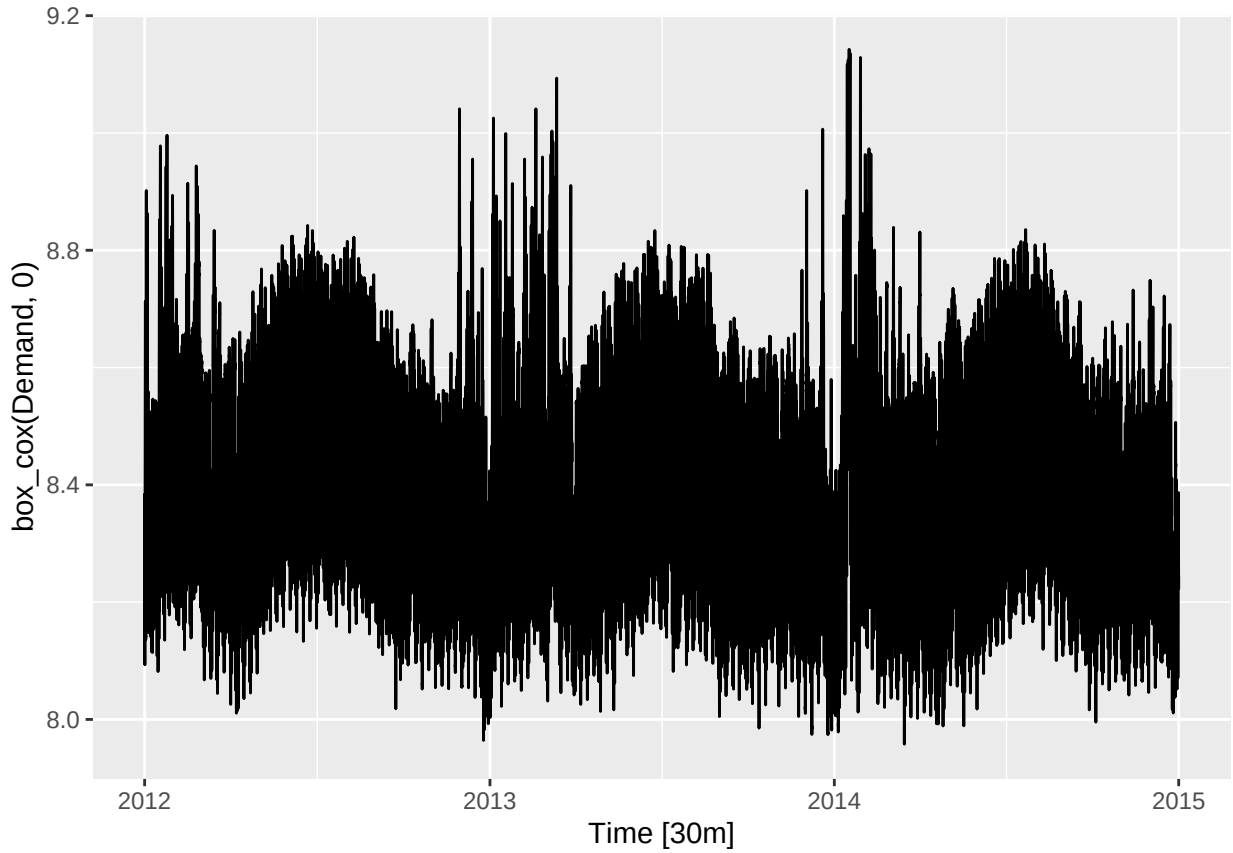
```
## # A tibble: 1 x 3
##   Animal                      State    lambda_guerrero
##   <fct>                       <fct>              <dbl>
## 1 Bulls, bullocks and steers Victoria         -0.0446
```

```
#Pretty close, guerrero suggests lambda=-0.0446. This is close enough to zero,
# so it is probably best to just use a log transformation (allowing better interpretations).
```

```
# Victorian Electricity Demand
vic_elec %>%
  autoplot(Demand)
```

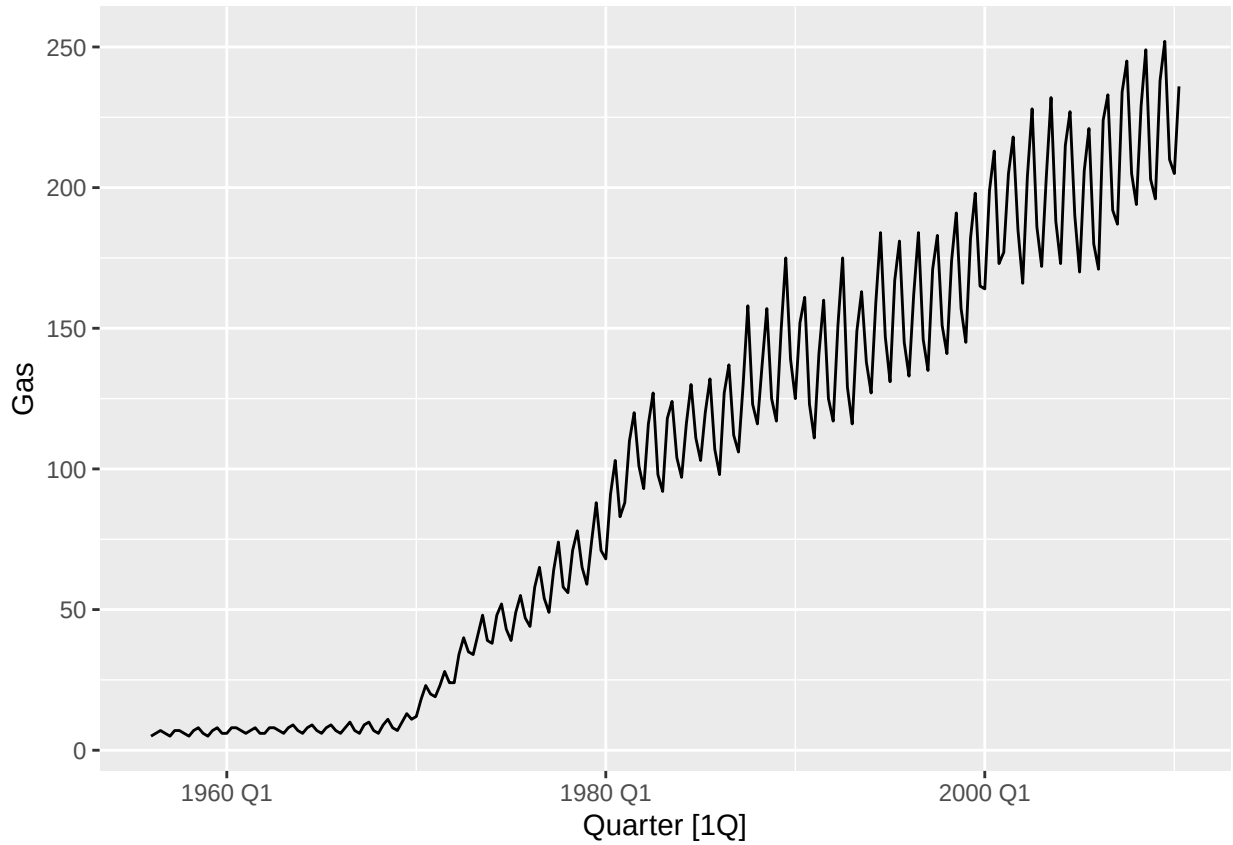



```r
#Seasonal patterns for time of day hidden due to density of ink. Day-of-week
# seasonality just visible. Time-of-year seasonality is clear with increasing
# variance in winter and high skewness in summer.

vic_elec %>%
  autoplot(box_cox(Demand, 0))
```

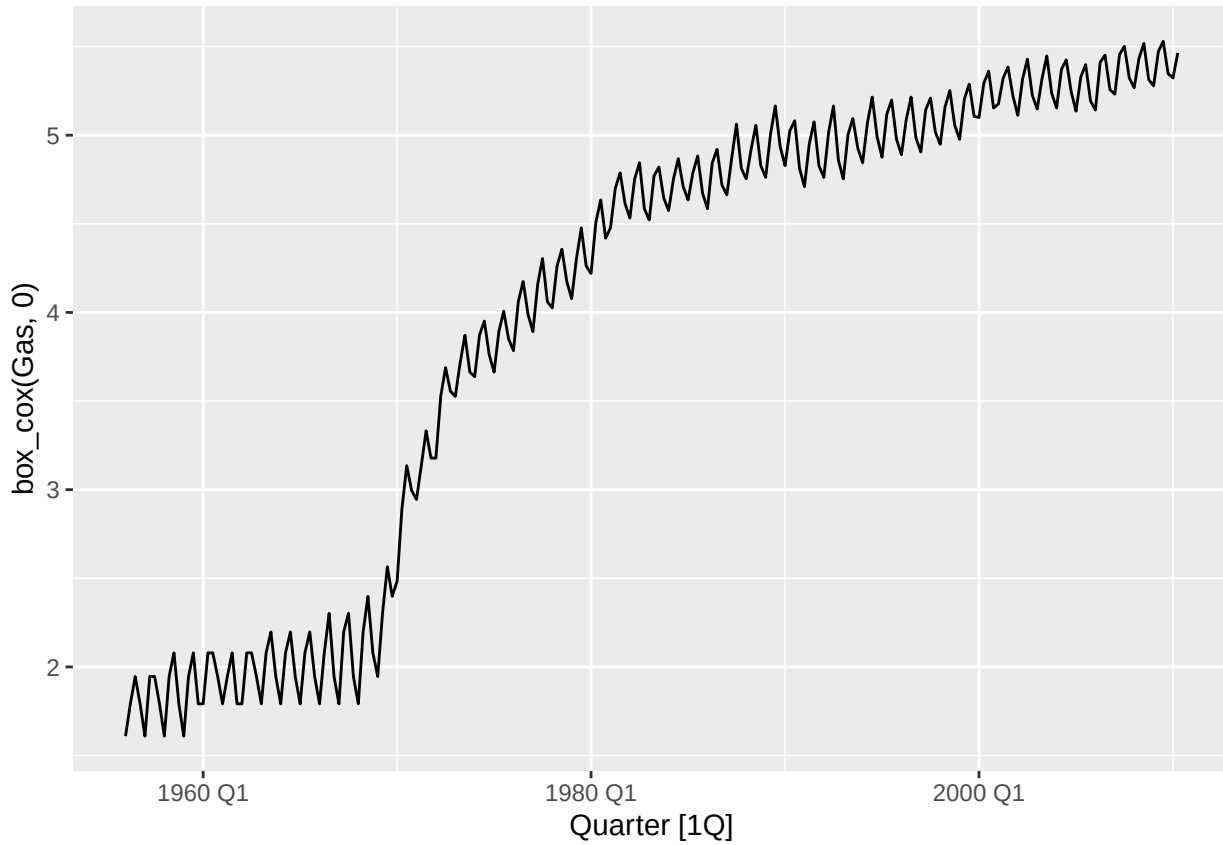



```
#A log transformation makes the variance more even and reduces the skewness.
#Guerrero's method doesn't work here as there are several types of seasonality.
```

```
#Australian Gas production

aus_production %>%
  autoplot(Gas)
```



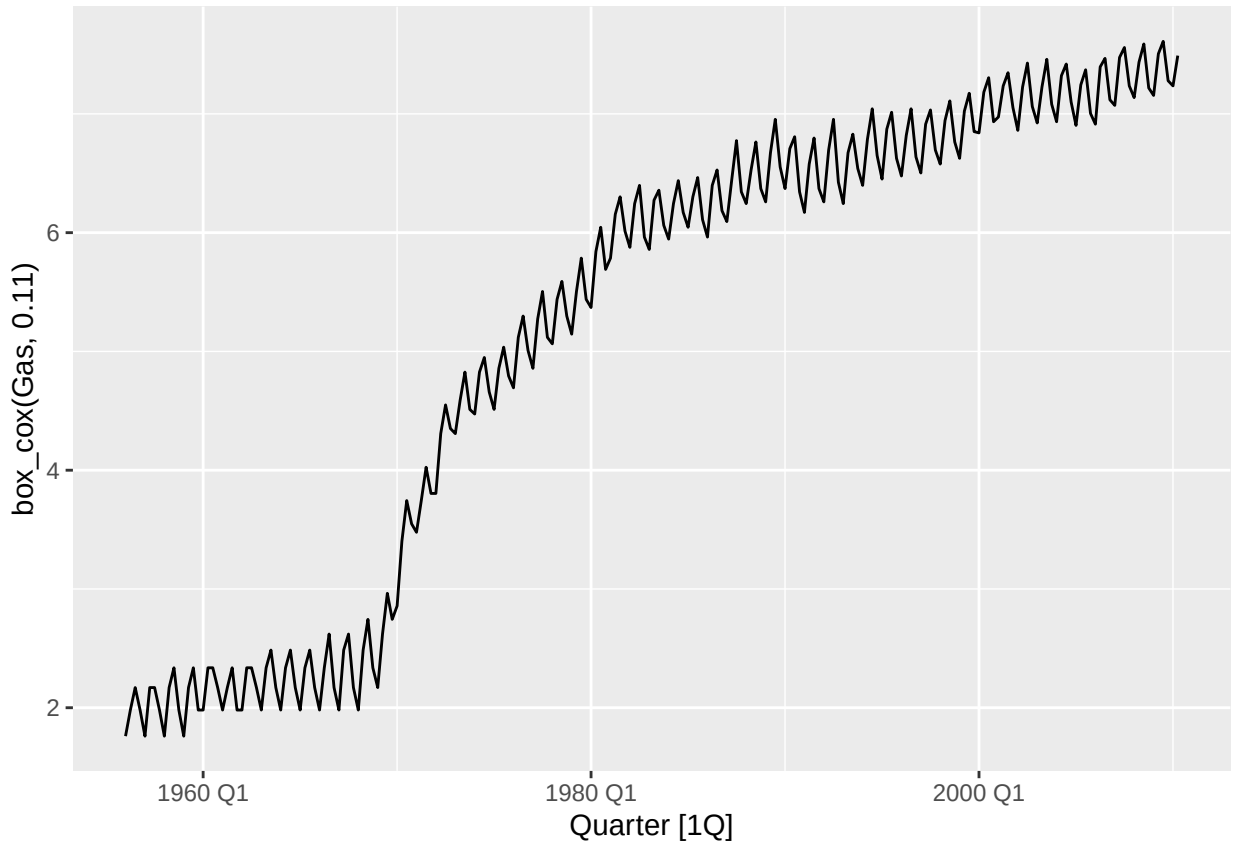

```
# Variation in seasonal pattern grows proportionally to the amount of gas
# produced in Australia. A transformation should work well here.

aus_production %>%
  autoplot(box_cox(Gas, 0))
```




```r
#A log transformation appears slightly too strong, where the variation in periods
# with smaller gas production is now larger than the variation during greater gas production.

aus_production %>%
  features(Gas, features = guerrero)
```

```
## # A tibble: 1 x 1
##   lambda_guerrero
##             <dbl>
## 1           0.110
```

```r
# Guerrero's method agrees by selecting a slightly weaker transformation. Let's see how it looks.

aus_production %>%
  autoplot(box_cox(Gas, 0.110))
```



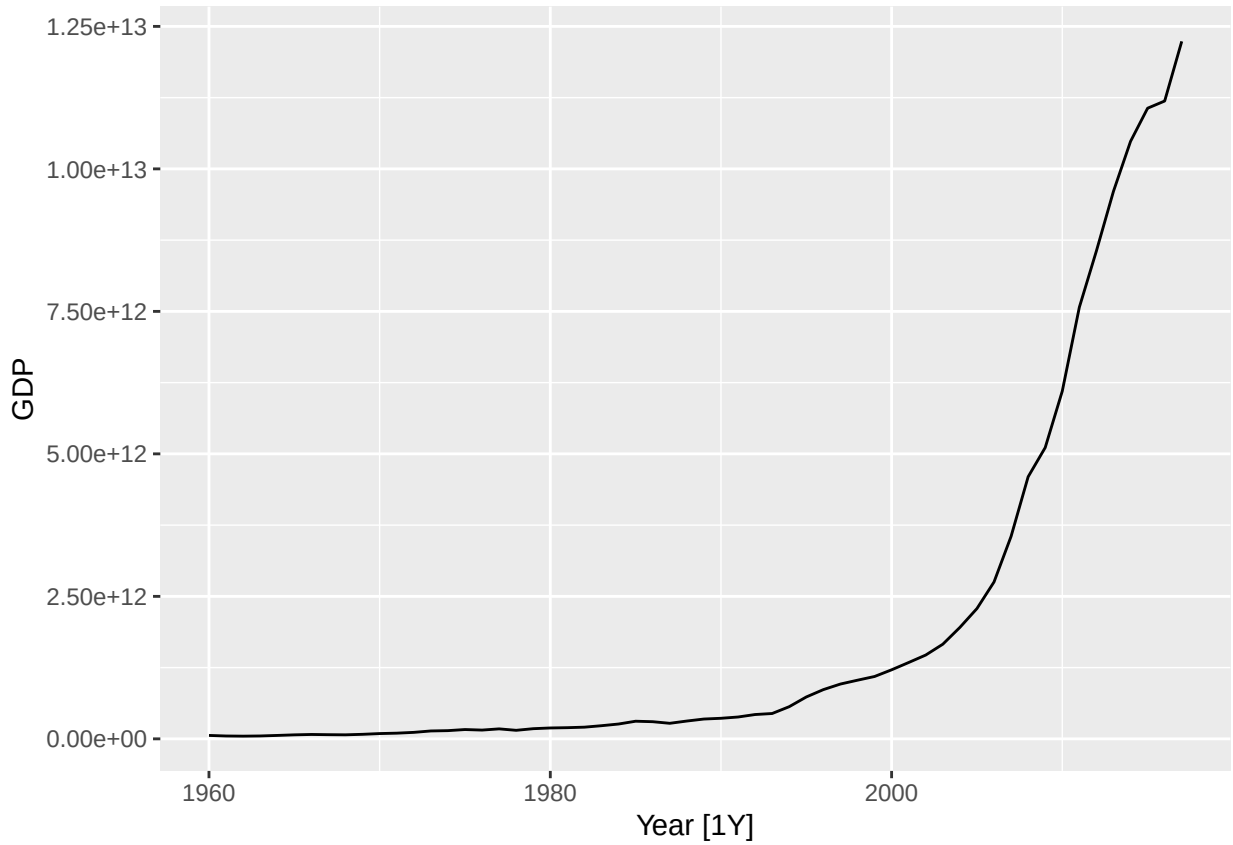

```
# Looking good! The variation is now constant across the series.
```

8.6

Forecast the Chinese GDP from the *global_economy* data set using an ETS model. Experiment with the various options in the *ETS()* function to see how much the forecasts change with damped trend, or with a Box-Cox transformation. Try to develop an intuition of what each is doing to the forecasts.

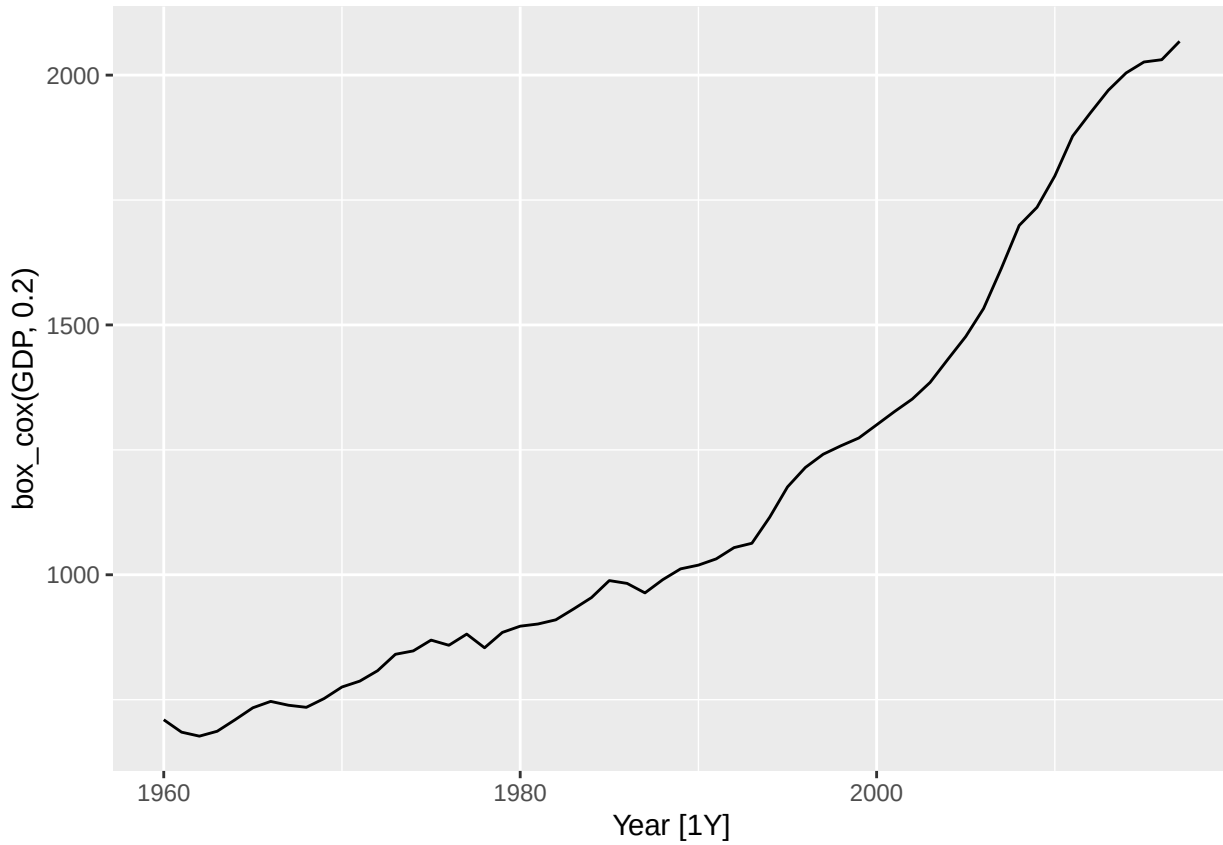
[Hint: use h=20 when forecasting, so you can clearly see the differences between the various options when plotting the forecasts.]

```
china <- global_economy %>%
  filter(Country == "China")
china %>% autoplot(GDP)
```




```
# It clearly needs a relatively strong transformation due to the increasing variance.
```

```
china %>% autoplot(box_cox(GDP, 0.2))
```




```
china %>% features(GDP, guerrero)
```

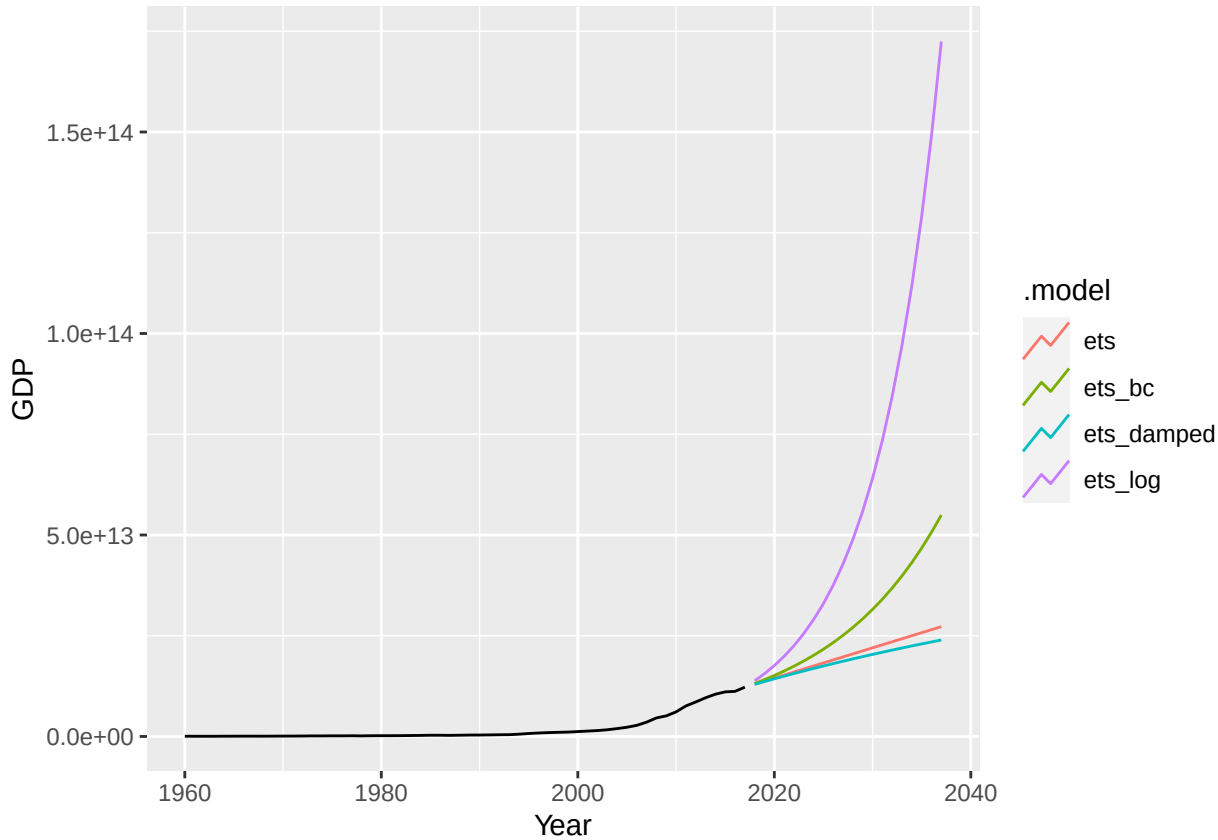
```
## # A tibble: 1 x 2
##   Country lambda_guerrero
##   <fct>             <dbl>
## 1 China           -0.0345
```

```
# Lambda = -0.034
```

```
fit <- china %>%
  model(
    ets = ETS(GDP),
    ets_damped = ETS(GDP ~ trend("Ad")),
    ets_bc = ETS(box_cox(GDP, 0.2)),
    ets_log = ETS(log(GDP))
  )

fit
```

```
## # A mable: 1 x 5
## # Key:     Country [1]
##   Country          ets    ets_damped       ets_bc      ets_log
##   <fct>        <model>       <model>      <model>      <model>
## 1 China   <ETS(M,A,N)> <ETS(M,Ad,N)> <ETS(A,A,N)> <ETS(A,A,N)>
```

```r
fit %>%
  forecast(h = "20 years") %>%
  autoplot(china, level = NULL)
```




```r
# The transformations have a big effect, with small lambda values creating big
# increases in the forecasts. The damping has relatively a small effect.
```

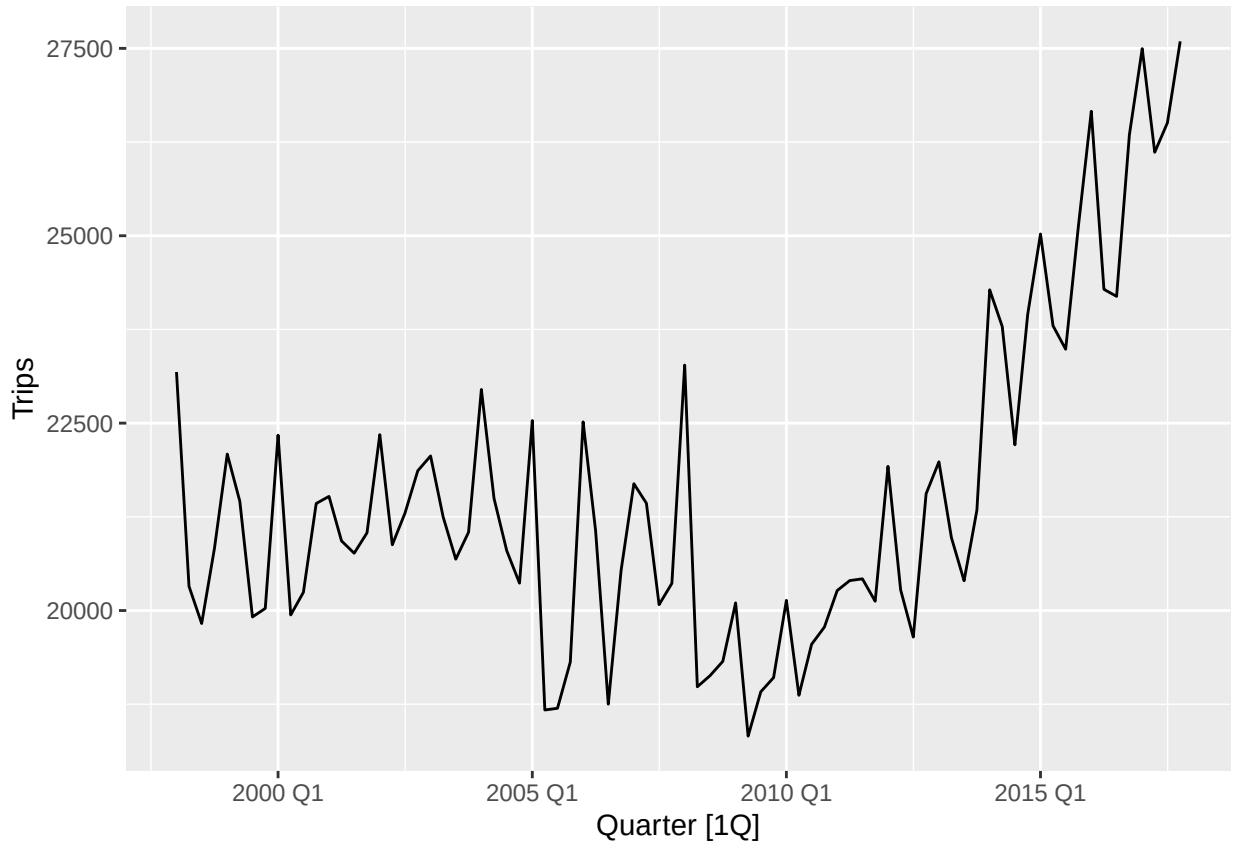
8.10

Compute the total domestic overnight trips across Australia from the tourism dataset.

a.

Plot the data and describe the main features of the series.

```r
aus_trips <- tourism %>%
  summarise(Trips = sum(Trips))
aus_trips %>%
  autoplot(Trips)
```

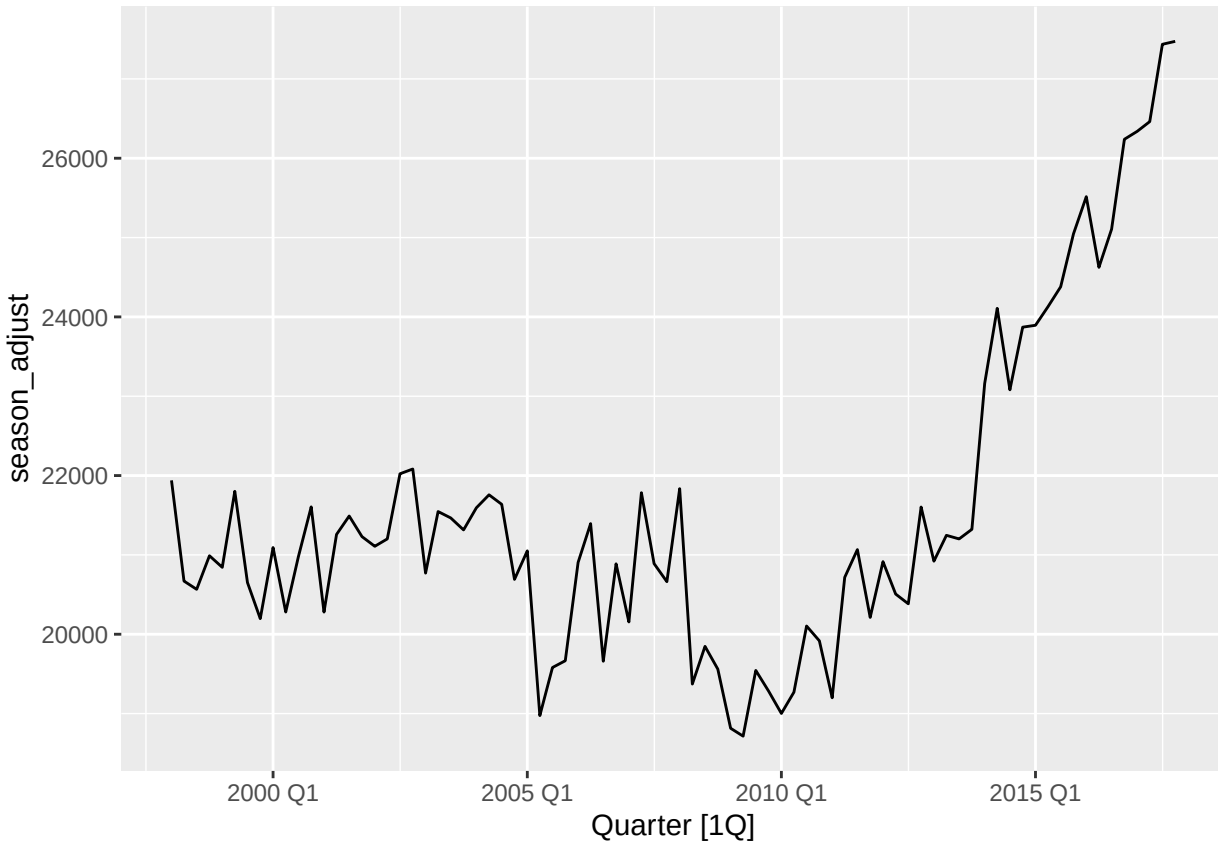



```
# The data is seasonal. A slightly decreasing trend exists until 2010,
# after which it is replaced with a stronger upward trend.
```

b.

Decompose the series using STL and obtain the seasonally adjusted data.

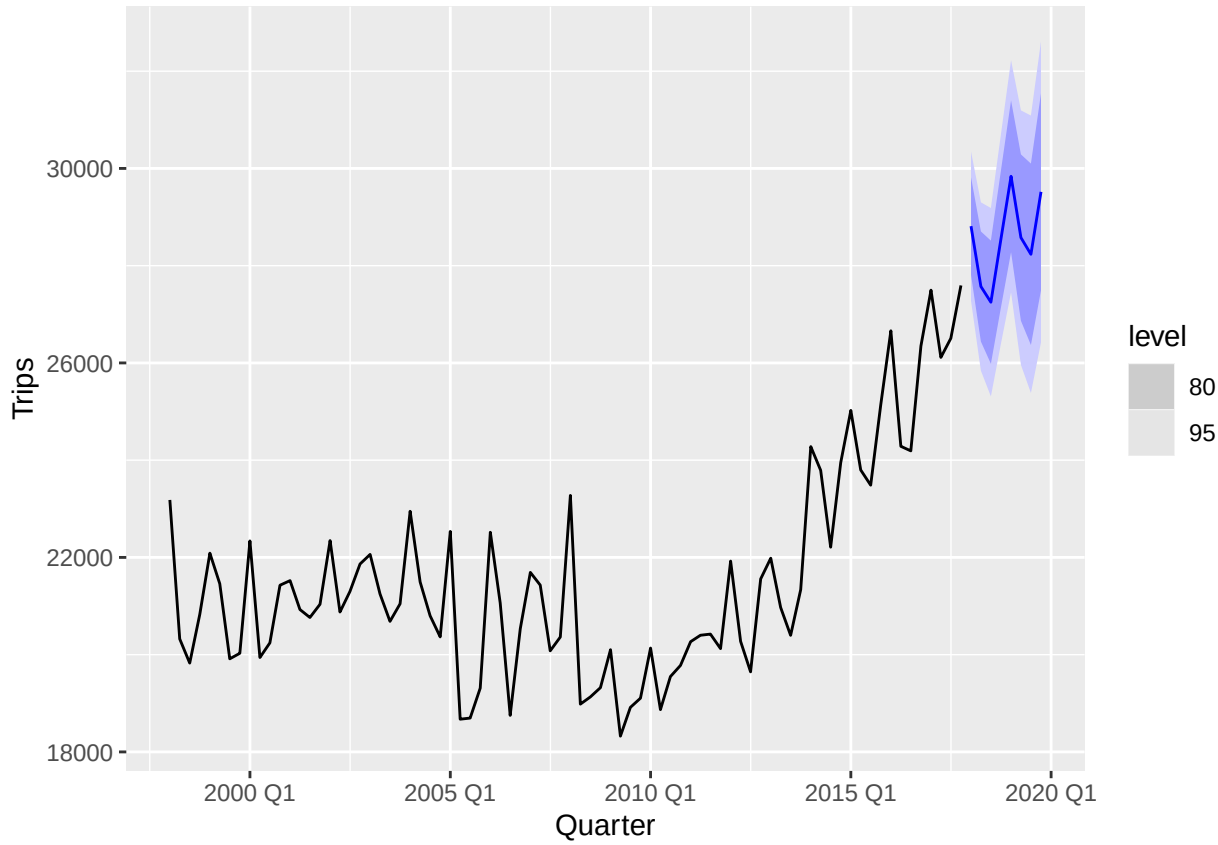
```
dcmp <- aus_trips %>%
  model(STL(Trips)) %>%
  components()
dcmp %>%
  as_tsibble() %>%
  autoplot(season_adjust)
```



c.

Forecast the next two years of the series using an additive damped trend method applied to the seasonally adjusted data. (This can be specified using *decomposition_model()*.)

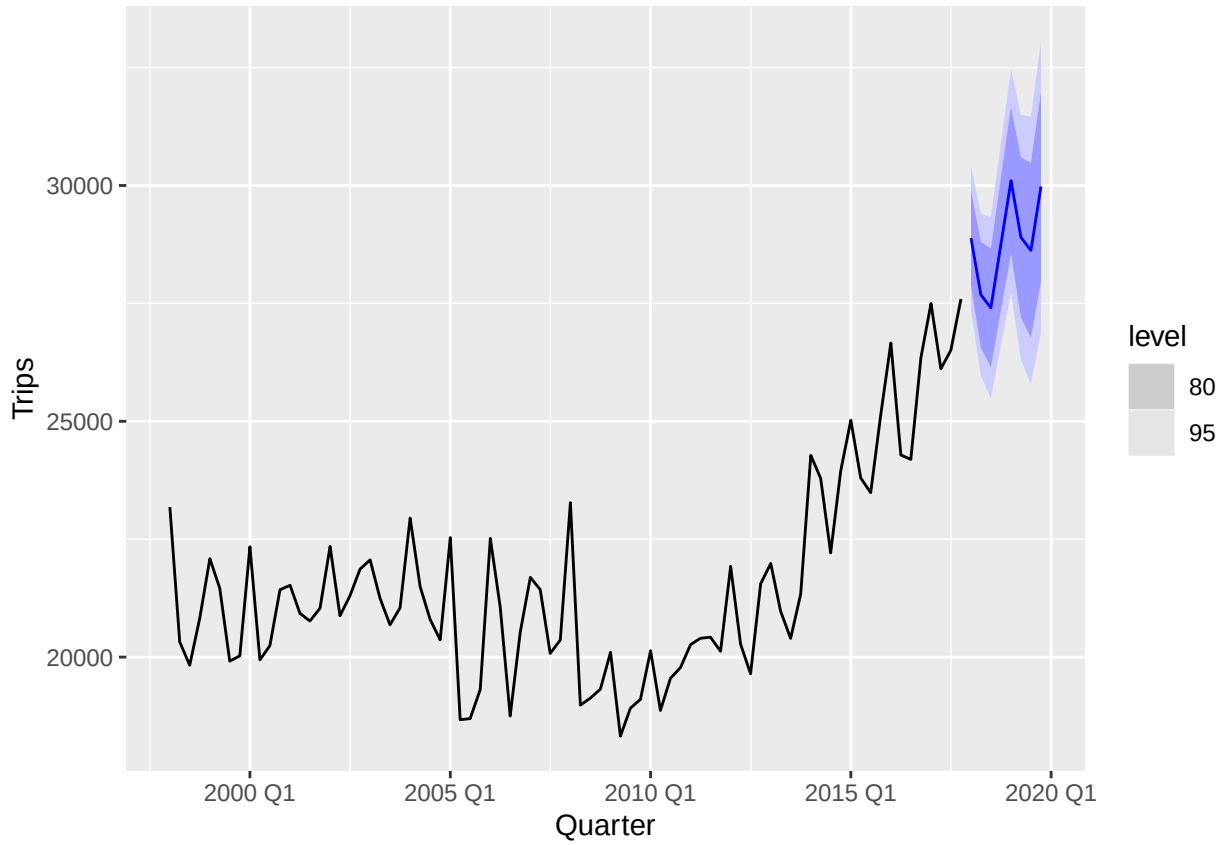
```
stletsdamped <- decomposition_model(
  STL(Trips),
  ETS(season_adjust ~ error("A") + trend("Ad") + season("N")) #Here: "A" means an additive term for the
  #trend elements. "N" means none, meaning no seasonality should be added. "Ad" means an additive dampe
)
aus_trips %>%
  model(dcmp_AAdN = stletsdamped) %>%
  forecast(h = "2 years") %>%
  autoplot(aus_trips)
```



d.

Forecast the next two years of the series using an appropriate model for Holt's linear method applied to the seasonally adjusted data (as before but without damped trend).

```
stletstrend <- decomposition_model(
  STL(Trips),
  ETS(season_adjust ~ error("A") + trend("A") + season("N")) #Here: "A" means an additive term for the
  #trend elements. "N" means none, meaning no seasonality should be added.
)
aus_trips %>%
  model(dcmp_AAN = stletstrend) %>%
  forecast(h = "2 years") %>%
  autoplot(aus_trips)
```

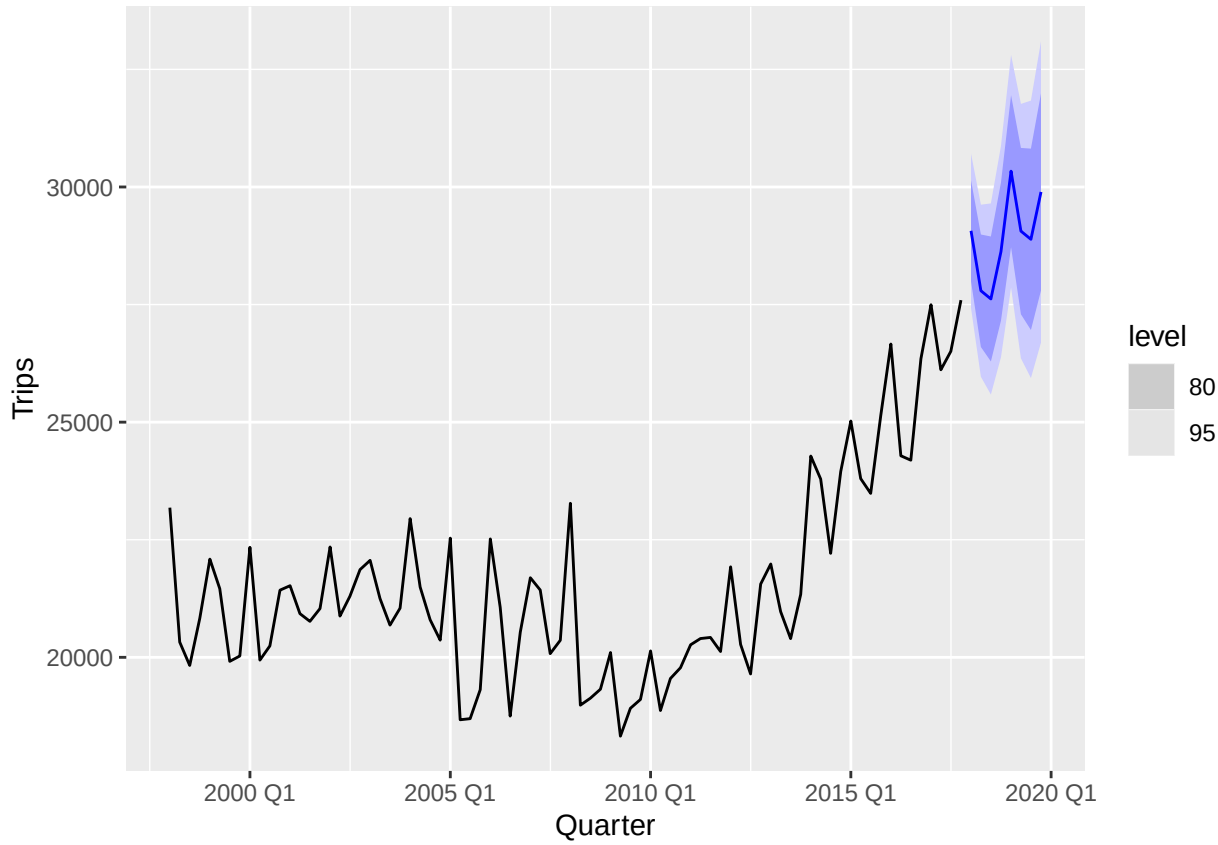



```
##no damped trend here
```

e.

Now use *ETS()* to choose a seasonal model for the data.

```
aus_trips %>%
  model(ets = ETS(Trips)) %>%
  forecast(h = "2 years") %>%
  autoplot(aus_trips)
```



f.

Compare the RMSE of the ETS model with the RMSE of the models you obtained using STL decompositions. Which gives the better in-sample fits?

```
fit <- aus_trips %>%
  model(
    dcmp_AAdN = stletsdamped,
    dcmp_AAN = stletstrend,
    ets = ETS(Trips)
  )
accuracy(fit)
```

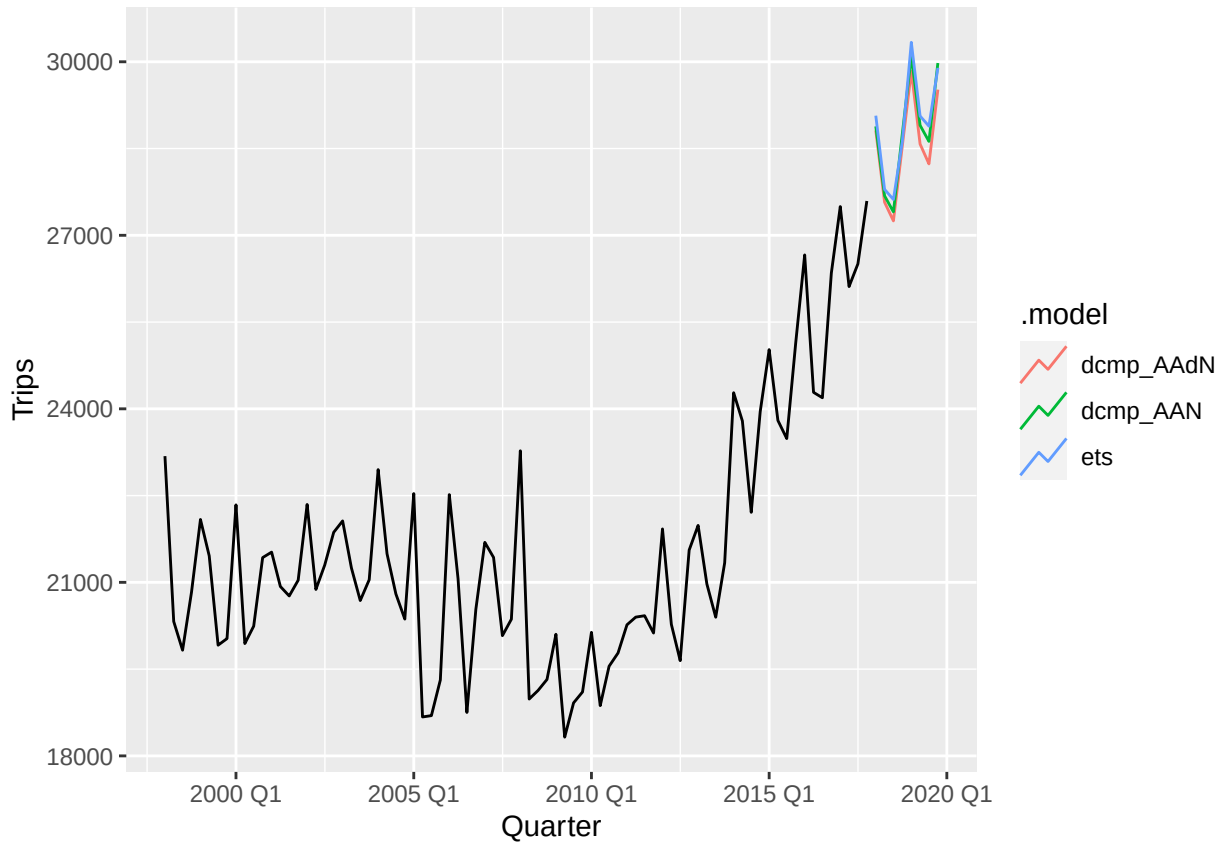
```
## # A tibble: 3 x 10
##   .model    .type       ME  RMSE   MAE   MPE  MAPE  MASE RMSSE     ACF1
##   <chr>     <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>    <dbl>
## 1 dcmp_AAdN Training 103.   763.  576. 0.367  2.72 0.607 0.629 -0.0174
## 2 dcmp_AAN  Training  99.7  763.  574. 0.359  2.71 0.604 0.628 -0.0182
## 3 ets       Training 105.   794.  604. 0.379  2.86 0.636 0.653 -0.00151
```

```
# The STL decomposition forecasts using the additive trend model, ETS(A,A,N),
# is slightly better in-sample. However, note that this is a biased comparison
# as the models have different numbers of parameters.
```

g.

Compare the forecasts from the three approaches? Which seems most reasonable?

```
fit %>%
  forecast(h = "2 years") %>%
  autoplot(aus_trips, level = NULL)
```



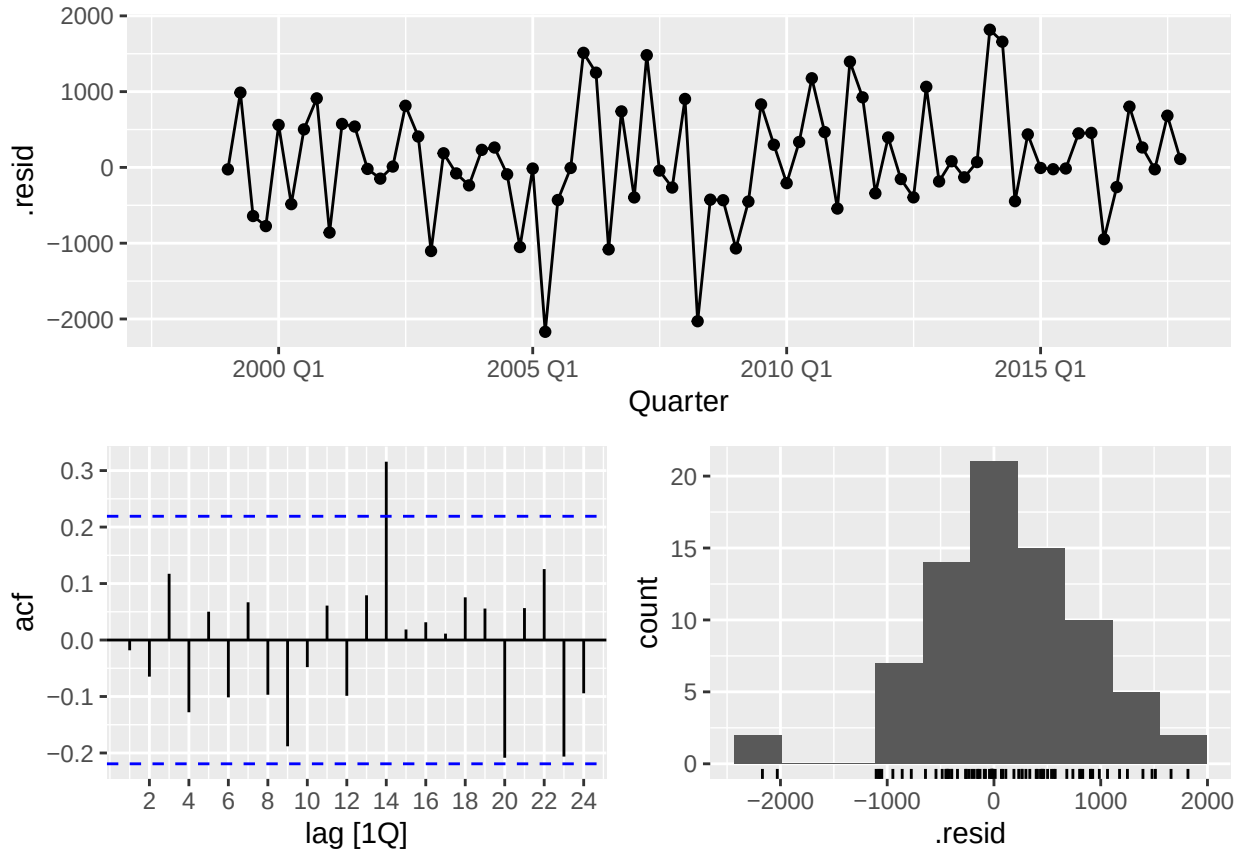

```
# The forecasts are almost identical. So I'll use the decomposition model with
# additive trend as it has the smallest RMSE.
```

h.

Check the residuals of your preferred model.

```
best <- fit %>%
  select(dcmp_AAN)
augment(best) %>% gg_tsdisplay(.resid, lag_max = 24, plot_type = "histogram")
```

```
## Warning: Removed 4 rows containing missing values (`geom_line()`).

## Warning: Removed 4 rows containing missing values (`geom_point()`).

## Warning: Removed 4 rows containing non-finite values (`stat_bin()`).
```




```r
augment(best) %>%
  features(.innov, ljung_box, lag = 24, dof = 4)
```

```
## # A tibble: 1 x 3
##   .model   lb_stat lb_pvalue
##   <chr>      <dbl>     <dbl>
## 1 dcmp_AAN    33.3    0.0311
```

```r
# The residuals look okay however there still remains some significant
# auto-correlation. The null of the LB test is rejected. This is mainly due to
# the large spike at lag 14 which from a modelling perspective we would probably
# ignore. Try testing for joint auto-correlation up to length 12 and see whether
# the test results change.
```

8.15

Show that the point forecasts from an ETS(M,A,M) model are the same as those obtained using Holt-Winters' multiplicative method.

Point forecasts from the multiplicative Holt-Winters' method:

$$\hat{y}_{t+h|t} = (\ell_t + hb_t)s_{t-m+h_m^+} \tag{1}$$

An ETS(M,A,M) model is given by:

$$y_t = (\ell_{t-1} + b_{t-1})s_{t-m}(1 + \epsilon_t)$$
$$\ell_t = (l_{t-1} + b_{t-1})(1 + \alpha\epsilon_t)$$
$$b_t = b_{t-1} + \beta(\ell_{t-1} + b_{t-1})\epsilon_t$$
$$s_t = s_{t-m}(1 + \gamma\epsilon_t)$$

So y_{T+h} is given by:

$$y_{T+h} = (\ell_{T+h-1} + b_{T+h-1})s_{T+h-m}(1 + \epsilon_{T+h-m})$$

Replacing ϵ_t by zero for t > T, and substituting in from the above equations, we obtain:

$$\hat{y}_{T+h} = (\ell_{T+h-2} + 2b_{T+h-2})s_{T+h-m} \tag{2}$$

Repeating the process a few times leads to:

$$\hat{y}_{T+h} = (\ell_{T+h-2} + hb_T)s_{T+h-m} \tag{3}$$

Doing the same thing for s_{T+h-m} gives:

$$\hat{y}_{t+h|t} = (\ell_t + hb_t)s_{t+h_m^+-m} \tag{4}$$

as required.